## Backend Frameworks and Key Considerations

*Core functions*

So we must cover

1. **Authentication Service:**
   Implement a passwordless login mechanism that issues short-expiry, single-use email codes for user sign-in. Includes token management and session security.

2. **User and Role Management:**
   Build a comprehensive database schema that models users, their roles (student, mentor, supervisor, administrator), and relationships with groups and resources. Role-based permissions strictly govern access.

3. **Tracks and Groups Structure:**
   Model mentoring tracks based on geographic regions and other criteria. Support dynamic creation and management of mentoring groups tied to these tracks.

4. **Chat Service:**
   Develop a scalable chat backend capable of handling real-time messaging and file transfers within groups. The service may be custom-built or integrate with a provider, depending on evaluation.

5. **Resource Library Storage:**
   Handle secure upload, storage, and retrieval of documents and resources with role-based visibility controls.

**API endpoints**

Provide a robust RESTful API layer exposing endpoints for:

- User profile import/export (including syncing with Airtable profiles)
- Retrieving and managing group memberships and chat messages
- Account activation/deactivation and status management

**External Integrations:**

Enable synchronisation with Airtable for profile management and Humanitix for event data, ensuring data consistency across systems.

## Language

First we gotta pick our language which will inform the frameworks that will be available for us

### 1. Python

| Pros | Cons |
|---|---|
| - Most of us pretty familiar with python<br>- Many libraries available for creating APIs as well as for analysis which might be helpful for stream 3<br>- Code is readable for Will and Vik for deployment | - A bit slower compared to Java and js since it isn't compiled but given the project performance besides chat functionality doesn't seem massive |

*Frameworks*

FastAPI

| Pros | Cons |
|---|---|
| - Ideal for REST APIs and microservices<br>- Supports JWT, OAuth2, MongoDB, SQL, async tasks<br>- Supports ORM pretty cool its basically like we can use functions to query DBs instead of actual SQL queries<br>- Easy integration with analytics libraries (pandas, NumPy)<br>- Modular, easy to test and deploy with Docker<br>- Auto generating documentation that is visible to anyone | - Not full-stack — front-end, templates, and admin interface need to be built separately<br>- Smaller ecosystem<br>- Some complex plugins are newer and less mature |

Django

| Pros | Cons |
|---|---|
| - Probs the most thorough framework and common in more professional and enterprise scale applications<br>- UI and admin panel available for CRUD operations and editing data<br>- Built in auth and user management | - Highest learning curve<br>- Slowest of the frameworks but kinda considered the most reliable - but still not a massive consideration<br>- Ngl watched 8 min rundown video got lost on the 5th second<br>- More for full-stack like say next.js- usually a plus but we might be |

| | |
|---|---|
| | negatively impacting other groups |

Flask

| Pros | Cons |
|---|---|
| - Most lightweight framework<br>- Definitely the easiest and something we're most familiar with<br>- Something the clients are familiar with from the description<br>- Good for small scale projects<br>- Flexible architecture | - Synchronous by default so may be more difficult to handle concurrent requests<br>- No built in ORM like FastAPI and Django<br>- Requires setup for additional features such as ORM, auth and input validation which is provided in django and fastapi<br>  - Also have to make more decisions for architecture as django and fastapi have more conventions |

**2. Java**

| Pros | Cons |
|---|---|
| - Still something that we're kinda familiar with<br>- Good performance<br>- Strong typing and maintainability<br>- Good support for security<br>- Lots of libraries available | - Slower development as it needs more setup - boilerplating<br>- Harder to develop asynchronous programs but is possible<br>- Falls to python in analytics support |

Really mainly one framework

Springboot

| Aspect | Pros | Cons / Considerations |
|---|---|---|
| **Development Speed** | Auto-configuration, embedded server, annotations reduce boilerplate setup | More verbose than Python; slower to prototype new features |
| **Scalability & Concurrency** | Handles large user bases; supports reactive programming and async | Async/reactive setup is more complex than Python async/await |
| **Security** | Robust security with Spring Security; JWT, OAuth2, role-based access | Requires understanding Spring Security concepts; more configuration needed |
| **Maintainability** | Enforces clean, layered structure (controllers → services → repositories); type-safe | Steeper learning curve for new developers; annotations-heavy |
| **Ecosystem & Integrations** | Mature ecosystem; libraries for DB, caching, messaging, monitoring; easy integration with external systems | Heavier framework; overkill for lightweight or rapid-prototype projects |
| **Analytics & Data Handling** | Can integrate with data libraries; good for enterprise analytics | Python has richer ecosystem for analytics, dashboards, and rapid data processing |
| **Front-end / API Documentation** | Can generate Swagger/OpenAPI documentation with Springdoc | Slightly more setup required compared to FastAPI's automatic docs |
| **Real-time Features** | Supports chat/notifications using WebFlux or @Async | More complex to implement than Python async-native frameworks |

### 3. Javascript

| Pros | Cons |
|---|---|
| - Can allow for a full stack javascript project<br>- Asynchronous<br>- Big ecosystem<br>- Lightweight and quick frameworks<br>- Good for real time applications | - Not something we have generally learnt in uni but in saying this it is learnable<br>- Less mature analytics support |

*Frameworks*

Express.js

| Pros | Cons |
|---|---|
| - Lightweight and easy support for db use<br>- Large ecosystem for authentication, JWT, emailing, validation etc<br>- Realtime features with Socket.io for messaging<br>- Fast for developing prototypes | - Very free structure so we would have to do a lot of planning but still something we'd have to do regardless<br>- Manual set up for auth, validation, security<br>- Less support for analytics |

NestJS

Pretty similar to express but requires more boilerplates and set up

**OVERALL OPINION**

Tbh I am leaning more towards say Python and FastAPI. It pretty much meets all the requirements of the project whilst giving the lowest learning curve possible. Has lots of support for analysis and creating APIs.

## Database options

So basically for this we got two main options SQL or NoSQL database

Tbh i have been looking at this and i think an SQL database would be pretty suitable. The fields and requirements aren't very complex, could think of it as a simple registry for users and chats and could be managed by a relational db. I think it'll be helpful for analytics too just since the relational structure will make querying easier. I did look into like chat storage too and apparently even like 1 mil entries a month is easily handled by say postgres. Overall also i think its something we are all most familiar with or at least the most approachable with each others support.

## Cloud Hosting and Services options

The two big boys are AWS and Azure. These aren't things that personally i haven't had too much work with but ill try give the best run down.

*AWS*

Amazon's cloud computing platform with a range of services including

| Service | Purpose |
|---|---|
| **EC2** | Virtual servers to run your applications (like a normal computer in the cloud). |
| **RDS** | Managed relational databases (Postgres, MySQL, etc.). |
| **S3** | Object storage for files/resources (documents, PDFs, images). |
| **Lambda** | Serverless computing – run code without managing servers. |
| **CloudWatch** | Monitoring, logging, and alerts. |
| **ECS / EKS** | Container orchestration for Dockerized apps. |

- Also good note here i think S3 if we go towards AWS is something we'll definitely be looking at for the resource library
    - Pretty much this is a separate db where we can store our larger attachments like files, images pdfs and stuff
    - To retrieve these S3 actually will provide URLs to access the data and can be used for lazy loading

*Azure*

Microsoft's cloud computing platform and honestly pretty similar to AWS. i think this may be a good consideration as it highly integrated with the Microsoft ecosystem which Biotech im pretty sure uses. He mentioned that they use sharepoint and pretty sent all our stuff via microsoft stuff. So might have to ask him about this. But tbh idk how it actually is integrated with their services.

| **Service** | **Purpose** |
|---|---|
| **App Service** | Managed platform to host web apps (like FastAPI). |
| **Azure Database for PostgreSQL** | Managed Postgres database. |
| **Blob Storage** | Object/file storage for resources. |
| **Functions** | Serverless code execution. |
| **Monitor / Application Insights** | Logging, performance tracking, and analytics. |
| **AKS / Container Instances** | Container orchestration (Docker / Kubernetes). |

**OVERALL OPINION**

Tbh this decision i think is more dependent on what biotech uses outside of the platform. It seems like they use the microsoft ecosystem so Azure might be more helpful? Azure has a lot of docs and prebuilt workflows to connect to it. But still honestly AWS has very similar and actually a broader scope of services which might be better for scaling. Idk if we rlly have to think this far rn but i guess its up for consideration. Cost i guess isnt as much as an issue too for consideration they did mention they do have grants available for either cloud provider.

## CI/CD

Tbh this isn't my forte, i did manage jenkins in that other project but totally up to suggestions for this one. I think this is also a service provided by the cloud providers too? Do need to look into this further.

## Summary

Ultimately these are still suggestions and honestly its very brief but i think just given a quick run down on the main options i think i would suggest.

Language and framework: Python, Django
Database: Azure postgres
Cloud provider: leaning a bit more to Azure but have to clarify more with our goat Will

**Decision:**
**Tech Stack:**

| | |
|---|---|
| Framework | Django + Django REST Framework (DRF), Channels for WebSockets |
| DB | PostgreSQL - we can use JSONB data type if we need as well<br>we'll use psycopg3.1.8 (aka psycopg) to connect django to our postgres db |
| Cloud | Azure |
| Storage | Azure Blob Storage |
| Docs | drf-spectacular (creates docs for DRF endpoints for OpenAPI) |
| Auth | OAuth, JWT |
| Infra | Docker Compose, GitHub Actions CI |
| Cache | Redis (if we have time + need to do this) |

**Django + DRF**
This is a Python web framework, DRF is our focus as the backend team. Theres a lot of tools - its focus is to make shipping an MVP quicker which is our goal.

Out of the box:
- Routing → maps urls to python functions
- ORM → we can define tables as Python classes and we can auto generate queries etc. makes db a lot easier.
- Auth and Sessions → has a lot of tools and support for this too
- Admin panel → theres a web accessible UI panel to CRUD models which is useful for testing and debugging. like H2 if you've used H2 before - or like using pgadmin
- templating engine → not really relevant to us
- has a lot of security defaults - CSRF, XSS, cookies, etc etc. this saves **A LOT** of time.

**DRF**
this is an extension of Django which helps us to **quickly** make **quality** APIs.

with DRF it is easy to:
- serialise: convert between python objects and JSON (and validate)
- viewsets + routers → stuff which we can use to make CRUD endpoints.
- Auth and perms: uses users/groups which makes it easy to enforce access control at the API level
- browsable api DOCS
- a lot of built in stuff like throttling, pagination, filtering etc.

**how django + drf maps to our workstream**
- passwordless login + sessions: django provides auth and session management - we can also add features to use one-time codes instead of passwords

- role based access (student, mentor, supervisor, admin) → django has groups/permissions built it - and we can use DRF to enforce these groups on endpoints.
- groups/tracks/memberships - easy to make relational models like these in Djangos ORM.
- chat backend - easy to use Django Channels (which is websockets) and enforce things like Auth and ORM (for chat history)
- resource library - django-storages is a library which makes it easy to implement and upload flow thru to Azure Blob
- integrations - gpt says something about this which i dont get yet but it should be easy with this tool called Celery

**nearest competitors**

- fastapi
    - theres a lot good about fast api, automatic docs, fast start up - but there is also less built in, we have to choose our own ORM e.g. sqlalchemy, our own admin, our own auth strategy (e.g. JWT vs sessions), background job system (e.g. integrating with humanitix) etc. its really good for pure small microservices, but assumes that the team is fluent in assembling pieces together.
- flask
    - it is an extremely minimal framework, but comes with nothing out the box e.g. auth, ORM, admin etc.
- express.js / node
    - great for consistency in fe/be dev, async by default etc. tbh idk why not but i feel like django comes with more shi
- theres a lot more scaffolding, more guardriails and less decisions to make - this is better suited for our team when we dont really know much.

**infra**

**environments**

we probs only need two envs: dev -> prod, we can skip staging until we start to integrate with other teams.
we set up two resources groups on Azure -> rg-dev, rg-prod and each has its own app, DB, cache and storage etc

**compute**

**Azure App Service**

- this is a managed container to run a Dockerised app
- we could also use Azure Container Apps (ACA) which are serverless containers, but thats a bit more complex.

**database**

we can host a cloud based DB using Azure → Azure Database for PostreSQL is a managed flexible server which manages a Postgres instance. works really well with our Django backend and ORM. each env will have its own DB.

**object storage**
fits for our file upload feature, we use Azure Blob Storage which are private containers and supports secure upload/download. it is easy to integrate with the django-storages library as well. our nearest alternative would be using S3 or keeping them in the DB (which is exxy)

**secrets and config**
so we also need to store API keys, passwords for DB and email creds as well, we can use Azure Key Vault for this. makes it a lot easier.

for local dev, we'll use a .env file as normal

**others**
theres also  support for:
- email for passworldless codes - Azure Communication Services
- Background jobs -> airtable/humanitix syncs, email etc → use Celery workers in a separate app service instance

**ci/cd**
we can use GitHub Actions or Azure DevOps Pipelines to run tests, build a Docker image, push to an Azure container, and deploy to AppService.

# Team topology (7 owners)

1. **Auth & Account State (Owner A)**

- **Scope:** Passwordless email codes; sessions/JWT; throttling; account activation/deactivation; `/auth/*`, `/users/{id}/activate|deactivate`, `/auth/me`.
- **Data:** `User`, `LoginCode`, basic role flags, account status.
- **Integrations:** Email sender (ACS/SendGrid).
- **DoD:**
  - Single-use codes (10–15 min expiry) + rate-limits + audit event on verify.
  - Session cookie + optional JWT flow documented; CSRF/CORS configured.
  - OpenAPI examples for FE; test fixtures for "student/mentor/supervisor/admin".

2. **Profiles & Directory (Owner B)**

- **Scope:** Student/Mentor/Supervisor profiles; interests; user search; import/export; `/users`, `/profiles/*`, `/interests`, `/users/import|export`.
- **Data:** `StudentProfile`, `MentorProfile`, `SupervisorProfile`, `InterestTag`.
- **DoD:**
  - Filterable directory (role, region, interests, text).
  - Upsert rules validated; CSV/JSON import/export tested with sample data.
  - Admin pages with useful filters.

3. **Tracks, Groups & Memberships (Owner C)**

- **Scope:** Tracks; groups; memberships; group snapshot; `/tracks`, `/groups`, `/groups/{id}/members`.
- **Data:** `Track`, `Group`, `GroupMembership`.
- **DoD:**
    - Membership invariants enforced (unique user per group; status transitions).
    - Group snapshot includes last message + unread counts (coordination with Chat).
    - Object-level permission helpers for “is group member?”.

4. **Chat & Realtime (Owner D)**

- **Scope:** WebSockets (Channels), REST fallback for messages; typing/read receipts; `/groups/{id}/messages`, `wss:///ws/groups/{id}`.
- **Data:** `Message` (+ delivery/read markers if you add them).
- **Dependencies:** Auth (WS auth), Groups (membership checks), Files (attachments).
- **DoD:**
    - Connect/auth/join; send/receive; persistence; pagination; cursor-based listing.
    - Load test (N users in a group); backpressure limits; basic reconnection strategy documented.

5. **Files & Resource Library (Owner E)**

- **Scope:** Pre-signed upload/download (Azure Blob SAS); resource listings; `/files/presign`, `/files/complete`, `/files/{id}`, `/resources`.
- **Data:** `File` (path, mime, size, scope, checksum).
- **Security:** Visibility scopes (role|group|track|public), size/MIME enforcement.
- **DoD:**
    - Upload flow e2e (request SAS → PUT to Blob → commit).
    - Time-limited download URLs; permission checks; optional AV hook documented.
    - Frontend copy-pasteable examples for multipart PUT.

6. **Integrations & Jobs (Owner F)**

- **Scope:** Airtable profile sync; Humanitix webhooks/backfill; Celery jobs + schedules; `/sync/*`, `/webhooks/humanitix`.
- **Data:** `ExternalIdentity` (+ last_synced_at, payload hashes), sync logs.
- **DoD:**
    - Idempotent upsert; retries/backoff; dry-run mode; admin status page with last run + counts.
    - Webhook signature verification; replay protection; error alerts.

7. **Platform, QA & Observability (Owner G)**

- **Scope:** CI/CD; environments; OpenAPI publishing; rate limits; logging/tracing; test data; security baselines.
- **Deliverables:**
    - GitHub Actions pipeline (build, test, generate schema, deploy), run migrations on deploy.

- Application Insights wiring + structured logs; Sentry (optional).
- DRF throttling (auth code requests), global exception handler shape `{code,message,details?}`.